

PhishGuard

An ML-Powered Phishing URL Detection System

Project Architecture & Implementation Report

Prepared for: End-of-Semester Project Submission

1. Introduction & Problem Statement

Phishing attacks remain one of the most prominent vectors for cyber threats, frequently bypassing traditional static blacklists by utilizing newly registered domains and sophisticated obfuscation techniques. Relying strictly on signature-based detection is no longer sufficient in a landscape where malicious actors can generate unique, disposable URLs at scale.

PhishGuard was developed to address this vulnerability by combining statistical Machine Learning classification with a real-time web architecture. Designed to mirror real-world Security Operations Center (SOC) workflows, the system operates primarily as a real-time Chrome browser extension that analyzes URLs on the fly. Results are synthesized and passed through a robust backend infrastructure, ultimately delivering structured threat insights accessible via a dedicated analyst dashboard.

2. System Architecture

The PhishGuard ecosystem is built on a modern, decoupled architecture ensuring high performance, scalability, and sub-500ms inference times. The system is divided into four distinct components:

1. The Brain: XGBoost Machine Learning Model

A serialized predictive model trained on a balanced dataset of legitimate and phishing URLs. It utilizes advanced lexical feature extraction to determine the probability of a URL being malicious without needing to load the actual web page content.

2. The Backend: FastAPI Server

A lightweight, high-performance REST API built with Python's FastAPI framework. It acts as the orchestration layer, receiving URLs from the client, processing the mathematical feature extraction, invoking the ML model for inference, and returning the confidence scores.

3. The Client: Chrome Browser Extension

A JavaScript and HTML-based browser extension utilizing Chrome's Manifest V3 API. It operates directly in the user's browser, securely capturing the active tab's URL and communicating asynchronously with the FastAPI backend to provide immediate visual threat feedback.

4. The UI: React Analyst Dashboard

A dedicated web portal for security analysts built using React and Vite. It serves as a centralized hub for manual URL scanning, displaying real-time AI confidence metrics, visual risk indicators, and simulated telemetry data for organizational threat monitoring.

3. Machine Learning Methodology

3.1 Dataset Preparation & Mitigation of Data Leakage

To train a robust model capable of achieving $\geq 95\%$ accuracy, the training data required rigorous curation. The primary source for malicious URLs was the well-known PhishTank database (`verified_online.csv`), which contains thousands of active, confirmed phishing links.

However, relying solely on this dataset introduces severe class imbalance—creating a "blind" model that assumes all URLs are malicious. To mitigate this and prevent overfitting, a large corpus of diverse, legitimate URLs (e.g., standard domains, deep paths, development repositories, and varied top-level domains) was generated and combined with the phishing data. This yielded a perfectly balanced dataset, allowing the model to accurately differentiate between safe and dangerous structural patterns.

3.2 Feature Engineering

Because malicious actors frequently change domain names, the model does not rely on memorizing specific strings. Instead, structural and lexical features are extracted dynamically using the `urllib.parse` and `re` libraries. The following features were engineered for the training pipeline:

Extracted Feature	Rationale for Inclusion
URL Length (<code>url_length</code>)	Phishing links often utilize excessively long URLs to hide the true domain from mobile or unobservant users.
Hyphen Count (<code>qty_hyphens</code>)	Attackers frequently use hyphens to mimic legitimate domains (e.g., <code>secure-login-bank.com</code>).
Dot Count (<code>qty_dots</code>)	Multiple subdomains are a common tactic to bypass basic domain reputation checks.
Presence of IP (<code>is_ip</code>)	Legitimate sites rarely use raw IP addresses in their URLs, making this a massive red flag for malicious infrastructure.

Extracted Feature	Rationale for Inclusion
Sensitive Words (has_sensitive_word)	A boolean flag checking for social engineering keywords in the URL string, such as "login", "secure", "bank", "verify", and "update".

3.3 Model Selection and Training

The **XGBoost (eXtreme Gradient Boosting) Classifier** was selected as the core algorithm for this project. XGBoost is highly optimized for tabular data and decision-tree ensembles, making it vastly superior to deep neural networks for purely lexical URL analysis in terms of both accuracy and computational efficiency.

The model was configured with `n_estimators=100`, a `learning_rate=0.1`, and evaluated using `logloss`. After splitting the data using an 80/20 train-test split, the model comfortably exceeded the project's 95% accuracy requirement. The final trained model was serialized using `joblib` into a `.pkl` file, ensuring the FastAPI backend could instantly load the pre-trained weights into memory without retraining upon server start.

4. Implementation & Deployment Execution

The implementation phase required strict separation of concerns across multiple environments:

- **Data Science Environment:** Jupyter Notebook was strictly utilized for data ingestion, feature engineering, model training, and serialization. This isolated the heavy computational tasks from the live application.
- **Backend Deployment:** The FastAPI server was developed in standard Python scripts using VS Code. To ensure environment consistency and prevent module execution errors, the server is initialized via the `python -m uvicorn` command. Cross-Origin Resource Sharing (CORS) middleware was explicitly configured to allow the Chrome extension and React dashboard to interface seamlessly.
- **Frontend Integration:** The React dashboard utilized Vite for rapid compilation. The Chrome extension's `manifest.json` was configured to request appropriate host permissions, ensuring that live tab queries could be forwarded to the local backend without triggering browser security blocks.

5. Conclusion

The PhishGuard project successfully demonstrates a full-stack, end-to-end integration of machine learning within a live web architecture. By shifting from simple string matching to algorithmic feature extraction, the XGBoost model accurately classifies complex, previously unseen URLs. Paired with the sub-500ms response time of the FastAPI server and the real-time intercept capabilities of the Chrome extension, PhishGuard represents a robust, highly scalable prototype for modern endpoint cybersecurity.